Object Oriented Programming: →

* Solving real world problems, and representing real world entities.

* Class
* Object / Instance / Reference
* Attributes / Properties
* Methods
* Code Reusability

Class ↓ Memory Block

Employee e1; → Object

name
emailID
salary
age

} Attributes / Properties

Class: → It is a blueprint to create objects / references / instances. It has no memory of its own (**). It is just used to instantiate objects.

Object: → It is an instance of a class. It is a key by which we can access all the properties (data members) inside the class. It is also called a reference.

Methods: → These are function that are created inside a class. They are called methods because they describe the behaviour of an object.

Attributes: → These are the common properties / parameters that are assigned to each object of the class.

Example: (Employee) → name, age, salary, etc.

Constructors: These are special methods which are used to instantiate objects.

* Rules to create constructors:
  → Same name as the class followed by ( ) Parentheses.
  → It has no return type.
  → It has to be public or else we cannot create objects.
  → There are two important types:

  (i) Default Constructor: It is generated by the compiler if we don't create it. — No-argument constructor.
  (ii) Parameterized Constructor: It is created by the user.

Name Clash: Name clashes between class attributes can be resolved in two ways:

(I) Writing different variable names:

```
// Attributes // Properties
string Name;
string emailID;
double salary;
string phoneNumber;
// Constructor
Employee(string name, string email, double s, string phone){
    Name = name;
    emailID = email;
    salary = s;
    phoneNumber = phone;
}
```

(II) Writing same names & using 'this' pointer.

```
// Attributes // Properties
string Name;
string emailID;
double salary;
string phoneNumber;
// Constructor
Employee(string Name, string emailID, double salary, string phoneNumber){
    this->Name = Name;
    this->emailID = emailID;
    this->salary = salary;
    this->phoneNumber = phoneNumber;
}
```

* Pillars:
  (I) Inheritance
  (II) Encapsulation
  (III) Polymorphism
  (IV) Abstraction

Book
  title
  ISBN
  author
  price
  bookDetails( );

* Normal
* Constructor